FORESHIFT LLC · CONFIDENTIAL · DEVELOPMENT PARTNER

ForeShift Web Application

Bubble Development Specification

Demand-Intelligence Platform · Detroit MVP · Model v8 (13 Zones × 9 Concepts)

For: Sliding Scale Technologies (Bubble development)

Platform: Bubble.io — app.foreshift.ai (application), foreshift.ai (marketing, Webflow)

Build order: Phase 1 (foundation) first · Phase 2 (AI + live layers) after or in parallel

Data status: **PLACEHOLDER / TEST DATA ONLY** — proprietary demand model & scoring logic are withheld pending provisional patent filing and counsel review

Scope: Whole application, phased. This document defines data model, screens, workflows, integrations, and phase boundaries.

> **IP boundary — read first.** This build uses **placeholder demand values** throughout. The real demand scores, the four-lever scoring formulas, and the per-venue calibration math are ForeShift proprietary IP and are **NOT** included in this spec. Build the machinery that *stores, retrieves, and displays* demand data against a placeholder dataset with the correct *structure*; the real values load later, after the IP gate clears. Nothing in this document should require knowledge of how a demand score is computed.

Contents

1 · Overview & Architecture

ForeShift gives restaurant operators a demand forecast for their location: how busy demand is likely to be, by day and by daypart, for their concept in their Detroit zone. The web app is where an operator signs up, is placed into a zone, buys a demand report, optionally subscribes to an AI intelligence pass, and logs how busy they actually were.

1.1 System pieces

Piece	Platform	Role
Application	Bubble.io (app.foreshift.ai)	Accounts, onboarding, demand report, subscriptions, operator dashboard, feedback capture, AI pass
Marketing site	Webflow (foreshift.ai)	Public marketing, lead capture (separate; not this spec)
Demand data	Loaded into Bubble DB	Zone × concept × day × daypart demand records (placeholder now, real later)
Zone geometry	External geocode + point-in-polygon	Assigns an operator's address to a zone at onboarding
AI (Phase 2)	Anthropic API via Bubble API Connector	Narrates demand in natural language, server-side
Events / weather (Phase 2)	Ticketmaster + weather API	Live signals pulled at query time to inform AI answers
Payments	Stripe (Bubble plugin)	Report purchase + AI pass subscription

1.2 Core data-flow principle (important for the AI)

> **The model produces the demand number; the app displays it; the AI narrates it.** The base demand score is a *fixed* value retrieved from the database (computed offline by ForeShift). The app never computes a demand score. In Phase 2, the AI receives the retrieved score plus live event/weather signals and writes a natural-language explanation — it **narrates** the model's output, it does not **generate** demand figures itself. This keeps the proprietary model outside the app and the AI honest.

2 · Phase Plan & Build Order

Build Phase 1 first — it is a complete, sellable product on its own. Phase 2 adds the AI intelligence layer and live data. Phase 2 may be built in parallel if capacity allows, but Phase 1 should not depend on Phase 2.

Capability	Phase	Notes
Accounts & auth	PHASE 1	Bubble native user accounts
Onboarding + zone auto-assignment	PHASE 1	Geocode → point-in-polygon → zone; concept suggest; confirm/adjust
Location Demand Report (static)	PHASE 1	Bands by day/daypart; placeholder values
Subscription / paywall (report purchase)	PHASE 1	Stripe; per-zone report product line
Operator feedback capture	PHASE 1	Busy-ness tap; store + export actuals
AI access pass (Claude narration)	PHASE 2	Anthropic API via API Connector; monthly tiers; query caps
Live events layer (Ticketmaster)	PHASE 2	Pulled at query time; feeds AI
Live weather layer	PHASE 2	Pulled at query time; feeds AI
Interactive operator dashboard	PHASE 2	Beyond the static report
Per-venue calibration engine	OUT	ForeShift-side, IP-gated — see §11

3 · Data Model (Bubble Data Types)

Create these Bubble Data Types. Field names are suggestions; types in parentheses are Bubble field types.

Zone

`name (text) · market_index (number) · character_description (text) · boundary_geojson (text — the polygon, for reference/display) · display_label (text) · slug (text)`

13 records (see Appendix A). Static reference data.

Concept

`name (text) · description (text) · operating_profile (text — e.g., "Midday + Dinner + Late Night") · slug (text)`

9 records (see Appendix A). Static reference data.

DemandRecord

`zone (Zone) · concept (Concept) · day (text: Mon–Sun) · daypart (text: morning/midday/dinner/late) · score (number — 0–150, PLACEHOLDER) · band (text: Minimal/Light/Moderate/High/Peak/Exceptional)`

This is the core demand table: 13 × 9 × 7 × 4 = **3,276 records**. Structure mirrors `model_bands.csv`. Values are placeholder until IP gate clears — the *schema* is what matters now.

Venue

`name (text) · address (text) · latitude (number) · longitude (number) · zone (Zone) · concept (Concept) · category_raw (text) · is_operator_venue (yes/no)`

Reference venue universe (~1,038 in-scope) + operator-created venues at onboarding.

Operator (extends Bubble User)

`venue (Venue) · zone (Zone) · concept (Concept) · onboarding_complete (yes/no) · subscription_status (text) · ai_pass_tier (text: none/single_zone/full_detroit) · report_access (list of Zone)`

FeedbackEntry

`operator (Operator) · venue (Venue) · date (date) · daypart (text) · busyness (text: Dead/Slow/Steady/Busy/Slammed) · actual_covers (number, optional) · predicted_band (text — snapshot at time of entry) · created_date (date)`

Collects operator actuals. App stores & exports these; it does NOT recalibrate the model (see §11).

Subscription

`operator (Operator) · product (text: report / ai_pass) · zone_scope (text) · stripe_subscription_id (text) · status (text) · current_period_end (date) · query_count_period (number)`

AIQuery (Phase 2)

operator (Operator) · question (text) · zone (Zone) · concept (Concept) · base_score_used (number) · events_context (text) · weather_context (text) · ai_response (text) · created_date (date)

Log of AI interactions; also used to enforce query caps.

4 · Data Loading & the Placeholder Dataset

> **Placeholder data.** ForeShift will provide a CSV with the correct structure (`zone, concept, day, daypart, score, band`) but with **placeholder score/band values**. Load it into the `DemandRecord` type via Bubble's CSV import (or the Data API). Build every screen and query against this. When the IP gate clears, ForeShift supplies the real CSV and it replaces the placeholder rows — no schema change required.

- **Zones & Concepts** — load the 13 zone and 9 concept reference records (Appendix A). These names are not proprietary and are final.
- **DemandRecord** — 3,276 rows, placeholder values, real structure.
- **Venue universe** — ForeShift provides the in-scope venue list (name, address, lat/lng, zone, concept) for onboarding lookups and zone density. This is reference data, not proprietary scoring.
- **Zone boundaries** — ForeShift provides GeoJSON polygons for the 13 zones (used by the point-in-polygon step in §5).

Naming consistency requirement. Zone names must match exactly across every data type and import (e.g., "Woodward Core", "Foxtown / Stadium District", "Downtown Detroit (Core)"). A mismatch will silently break joins between Venue, DemandRecord, and Zone. Use the exact strings in Appendix A.

5 · Onboarding & Zone Auto-Assignment

PHASE 1

The onboarding flow places a new operator into the correct zone from their address, with a confirm/adjust fallback. This is the entry point and must be smooth.

5.1 Flow

1. **Account creation** — email/password (Bubble native). Create the Operator record.
2. **Venue entry** — operator enters venue name + street address.
3. **Geocode** — pass the address to a geocoding service (Google Maps Geocoding API via API Connector, or Bubble's native geographic address field) → obtain latitude/longitude.
4. **Zone assignment (point-in-polygon)** — determine which zone polygon (from the loaded GeoJSON) contains the venue's coordinates. Set the Operator's zone. See 5.2 for implementation options.
5. **Concept suggestion** — suggest a concept based on the venue (if a matching Venue record exists in the reference universe, use its concept; otherwise present the 9 concepts for the operator to pick).
6. **Confirm / adjust** — show the operator the assigned zone (on a map) and suggested concept; let them confirm or manually change either. Store final zone + concept on the Operator and their Venue record.

5.2 Point-in-polygon — implementation options

Option	How	Trade-off
Server-side plugin / API	A geo library (e.g., a Turf.js-based endpoint or a plugin) tests the point against the 13 polygons	Most reliable; recommended
Bubble geographic + radius fallback	Nearest-zone-centroid if polygon test unavailable	Approximate; only as fallback
Precomputed lookup	ForeShift pre-assigns known venues; new addresses use the geo test	Fast for known venues

ForeShift supplies the 13 zone polygons as GeoJSON. If an address falls outside all 13 zones (out of the central-Detroit MVP area), show a "not yet covered" state and capture the lead.

> **Why this matters beyond onboarding:** the same zone geometry is the assignment engine. Get the name strings exact (Appendix A) so the assigned zone joins cleanly to DemandRecords.

6 · The Location Demand Report (Phase 1 core)

PHASE 1

This is the core Phase 1 product an operator buys — a demand report for their zone × concept, titled by neighborhood (e.g., "Corktown Demand Report").

6.1 What it shows

- **Header** — neighborhood-titled report name; zone character description; the concept.
- **Weekly demand pattern** — for the operator's zone × concept, the demand **band** for each day × daypart (Mon–Sun × morning/midday/dinner/late). Retrieved from DemandRecord.
- **Band display** — show the band label (Minimal → Exceptional) with a color scale. A grid (days × dayparts) is the natural layout.
- **Daypart breakdown** — the four dayparts (Morning 6–11, Midday 11–16, Dinner 16–21, Late 21–02) with their bands per day.
- **Read guide** — a legend explaining what each band means for staffing/prep (ForeShift supplies the band definitions; they are not proprietary).

6.2 How it's retrieved

Query `DemandRecord` where zone = operator's zone AND concept = operator's concept. This returns the 28 records (7 days × 4 dayparts). Group by day for the grid. **No computation in the app** — display the stored band. Values are placeholder until the real dataset loads.

> **Honest framing for the UI copy:** the report is *zone-level demand intelligence for the operator's concept* — a research-grounded baseline. Copy should say the report reflects expected demand patterns for the area and concept, refined over time. Do not imply it is a measured prediction of that specific venue's exact sales.

7 · Subscriptions, Paywall & Access Control

PHASE 1 (report) PHASE 2 (AI pass)

7.1 Products

Product	Type	Price (indicative)	Grants
Location Demand Report	Per-zone (one-time or recurring — confirm with ForeShift)	~$199/zone; ~$799 Detroit bundle	Access to that zone's static report
AI Access Pass — single zone	Monthly, cancel anytime	~$49/mo	AI queries for one zone (Phase 2)
AI Access Pass — full Detroit	Monthly, cancel anytime	~$99/mo	AI queries for all zones (Phase 2)

The AI pass is **decoupled** from the static report — an operator can buy either independently. Prices indicative; confirm final pricing with ForeShift before launch.

7.2 Implementation

- **Stripe** via Bubble's Stripe plugin. One-time charge for report purchase; Stripe subscription for the AI pass.
- **Access control** — on the Operator, maintain `report_access` (list of Zones they've purchased) and `ai_pass_tier`. Gate report pages and AI features on these. Use Bubble privacy rules so unpurchased data is not exposed client-side.
- **Query caps** — for the AI pass, track `query_count_period` on the Subscription; enforce a cap per billing period as cost control (cap value from ForeShift). Reset on `current_period_end`.

8 · Operator Feedback Capture

PHASE 1

Operators log how busy they actually were. This builds ForeShift's calibration dataset. **The app collects and exports; it does not recalibrate the model** (see §11).

8.1 Capture UI

- A quick **busy-ness tap**: five levels — **Dead · Slow · Steady · Busy · Slammed** — selectable per daypart for a given date.
- Optional **actual covers** numeric entry for operators who want to be precise.
- At entry time, snapshot the **predicted band** for that zone/concept/day/daypart onto the FeedbackEntry (so the comparison is preserved even if the model later changes).

8.2 What the app does with it

- **Store** each FeedbackEntry.
- **Display (optional)** a simple variance view to the operator — "you reported Busy; forecast was High" — for engagement only. This is descriptive, not a model change.
- **Export** — provide ForeShift an export (CSV/Data API) of FeedbackEntries. ForeShift performs calibration offline. **Do not build recalibration logic in the app.**

9 · The AI Access Pass (Phase 2)

PHASE 2

The AI pass lets an operator ask natural-language questions and get demand-aware answers. The AI **narrates** the model's stored demand plus live signals; it does not invent demand numbers.

9.1 Architecture

1. Operator asks a question (e.g., "How busy will this weekend be?").
2. Bubble backend workflow retrieves the relevant **DemandRecord(s)** (base bands for the zone/concept/days).
3. Bubble pulls **live context** — nearby events (§10) and weather (§10) for the relevant dates.
4. Bubble calls the **Anthropic Messages API** via the API Connector (server-side), passing: the retrieved demand bands, the event context, the weather context, and the question.
5. The system prompt instructs the model to **explain the demand using the provided numbers and context**, and explicitly **not** to fabricate demand values. The model returns natural-language text.
6. Display the response; log an AIQuery; increment the query counter.

9.2 API Connector setup

- Endpoint: Anthropic Messages API. **Server-side call** (API key stored in Bubble backend, never client-side).
- Model: a cost-appropriate Claude model (ForeShift will specify; Haiku-class was noted as margin-favorable). Confirm current model string with ForeShift at build time.
- Pass conversation + the retrieved demand/context as structured input. Keep `max_tokens` bounded for cost.
- The base demand figures come **only** from DemandRecord — the model must not be asked to compute them.

> **System-prompt principle (for ForeShift to finalize):** "You are given ForeShift's demand bands for this location and concept, plus live event and weather context. Explain what the operator can expect and why, using only the provided demand values and context. Do not invent or alter demand numbers." This keeps the proprietary model as the source of truth and the AI as the narrator.

10 · Live External Layers — Events & Weather (Phase 2)

PHASE 2

10.1 Events

- Source: Ticketmaster API (or equivalent) for Detroit venue events (stadiums, arenas, theaters).
- Pulled at **query time** for the relevant dates and the operator's zone vicinity.
- Used as **context for the AI** and, later, as input to ForeShift's event layer. In this build, events are passed to the AI as context — the app does not compute an event-adjusted demand score (that logic is ForeShift-side).
- Event relevance is by **proximity** (distance from the event venue to the zone), not zone membership.

10.2 Weather

- Source: a weather API (forecast for the relevant dates, Detroit).
- Pulled at query time; passed to the AI as context.

> **Note on scoring:** In this build, events and weather inform the AI's *narrative* only. Numeric adjustment of demand scores by events/weather is ForeShift-side and deferred until real operator outcome data exists to calibrate it. Do not implement demand-score math for events/weather in the app.

11 · Out of Scope (ForeShift-side / IP-gated)

> **Do not build these.** They are ForeShift proprietary IP and/or performed offline. The app must not contain this logic.

Item	Why out of scope
The demand scoring formulas (four-lever model, daypart overlap, Market Index derivation)	Core proprietary IP. The app displays stored scores; it never computes them.
Per-venue calibration engine (capture rate, per-venue curve, coupling)	Proprietary IP; runs offline at ForeShift. The app only *collects* the actuals that feed it.
Recalibration / model-updating from feedback	ForeShift-side. App exports FeedbackEntries; it does not adjust the model.
Event/weather numeric demand adjustment	ForeShift-side; deferred pending calibration data. App passes events/weather to the AI as context only.
The real demand values	Withheld pending patent filing. Build against placeholder data.

12 · Non-Functional Requirements

- **Data privacy** — Bubble privacy rules must prevent client-side exposure of demand data the operator has not purchased, and of other operators' feedback.
- **API keys server-side** — Anthropic, Ticketmaster, weather, and geocoding keys live in Bubble's backend; never exposed to the browser.
- **Caching of live data** — cache events/weather results server-side per zone/date to avoid repeated external calls and control cost/latency. (ForeShift note: reliability of external calls matters — do not call live APIs on every page load.)
- **Cost control** — enforce AI query caps; bound `max_tokens`; log AIQuery for monitoring.
- **Responsive** — operators will use this on mobile and desktop.
- **Placeholder-to-real swap** — the real demand dataset must be loadable without schema changes (same columns as the placeholder CSV).

A · Appendix — Reference Data & Screen Inventory

A.1 The 13 Zones (exact names — use verbatim)

Zone (exact string)	Market Index
Woodward Core	100
Foxtown / Stadium District	95
Downtown Detroit (Core)	93
Greektown / Casino District	92
Financial District	87
Midtown	83
Corktown	79
Eastern Market	72
New Center / North End	65
Riverfront / RiverWalk	61
Mexicantown	59
Core City / Woodbridge	58
Southwest Detroit	56

Market Index shown for reference; it is not proprietary scoring logic (it is a zone intensity label). The *demand scores* that depend on it are the withheld part.

A.2 The 9 Concepts (exact names — use verbatim)

Concept	Operating profile
Fine Dining	Dinner + Late Night
Upscale Casual	Midday + Dinner
Casual Dining	Morning + Midday + Dinner
Fast Casual	Morning + Midday + Dinner
Coffee Shop	Morning + Midday + Dinner
Breakfast / Brunch Cafe	Morning + Midday
Sports Bar	Midday + Dinner + Late Night
Cocktail Lounge	Dinner + Late Night
Neighborhood / Casual Bar	Midday + Dinner + Late Night

A.3 The four dayparts

Daypart	Window
Morning	6:00 AM – 11:00 AM
Midday	11:00 AM – 4:00 PM
Dinner	4:00 PM – 9:00 PM
Late Night	9:00 PM – 2:00 AM

A.4 The six demand bands

Band	Score range (0–150)
Exceptional	110–150
Peak	85–109
High	65–84
Moderate	40–64
Light	20–39
Minimal	0–19

A.5 Screen inventory

Screen	Phase	Purpose
Sign up / Log in	1	Auth
Onboarding wizard	1	Venue entry → zone → concept → confirm
Demand Report	1	Days × dayparts band grid
Purchase / checkout	1	Stripe report purchase
Feedback capture	1	Busy-ness tap + covers
Account / subscription	1	Manage plan, access
AI chat / query	2	Ask questions; see narration
AI pass upgrade / checkout	2	Subscribe to AI tier
Interactive dashboard	2	Richer demand + live signals

> **Build summary in one line.** Phase 1 delivers a complete self-serve product — operators sign up, are auto-assigned to a Detroit zone, buy a neighborhood Demand Report (built on placeholder data with the real schema), and log how busy they were. Phase 2 adds the AI Access Pass that narrates demand with live event and weather context. The proprietary scoring and calibration never enter the app; it stores, displays, and narrates values ForeShift supplies.